

Assignment 4 (Report)

Name: Gabriel Martinez

Course: WES 237A

GitHub: <https://github.com/Math140Instructor/wes237a/Assignment4>

Video: https://drive.google.com/file/d/10Zq3igmdRwtywdWoE_1mUOUxrPLRDTYH/view?usp=sharing

1. Objective

The goal of this assignment was to create a lightweight server that accepts incoming TCP connections on a specified port using Python's socket library and triggers the buzzer to sound for 0.5 seconds each time the server receives a message from a client.

2. Design Methodology

A top down design methodology was used to structure the implementation. The overall problem was decomposed into smaller, testable components:

1. I started by importing my pulse width modulation code from previous homework assignments and adapting it to work with the active buzzer.
2. I verified the .5 sec buzz by capturing the elapsed time
3. I then conducted a few headless tests on the passive buzzer, but ultimately decided to focus solely on completing the assignment requirements.
4. I implemented a button listener to ensure that each button responds to its own `threading.Event` when pressed.
5. I tested the socket server and client connections and made slight adjustments until I was satisfied.

TL;DR. Each component was verified independently and manually tested before integrating into the final workflow.

3. Workflow and Implementation

The strategy for this assignment was to learn how to control the buzzer, understand the socket API for listening for connections and sending messages, and ensure that the buttons operated independently by creating their own `threading.Event`, which I used as a flag for other threads to respond to when set.

4. Difficulties and Troubleshooting

The main difficulty was learning the socket API. After becoming comfortable with it, we encountered another issue while testing on a closed network because we were on different subnets. Once we corrected the subnet configuration, the connection worked properly.

5. Results and Analysis

The objectives of this assignment were successfully achieved. The primary goal was to use the PYNQ board to listen on a designated port, accept incoming messages, and trigger the buzzer upon receiving a message, which was verified to work as expected.

```
In [72]: from pynq.overlays.base import BaseOverlay
base = BaseOverlay("base.bit")
btns = base.btns_gpio
```

```
In [73]: %%microblaze base.PMODB

#include "gpio.h"
#include "timer.h"

// plugged my buzzer in upside down on ground pin so data pin D3 will activate
const unsigned DPIN=2;
gpio d_pin;

// set pin DPIN to write and the other pins to read
int init(){
    d_pin = gpio_open(DPIN);
    gpio_set_direction(d_pin, GPIO_OUT);

    // turn off other pins
    gpio parent = gpio_open_device(0);
    gpio in_pins = gpio_configure(parent, 1, 3, 1);
    gpio_set_direction(in_pins, GPIO_OUT);
    gpio_write(in_pins,0);

    return 0;
}

unsigned int write(unsigned int signal){
    if(!d_pin) init();
    gpio_write(d_pin, signal);
    return signal;
}

int pwm(unsigned int freq, unsigned int duty, unsigned int val){
    if(!d_pin)init();

    freq = freq==0 ? 1:freq;
    duty = duty%101;
```

```

    unsigned int us = 1000000; // work in micro secs (us)
    unsigned int T = (us / freq);

    unsigned int on_us = T * duty / 100.; // convert duty into percent
    unsigned int off_us = T - on_us;

    gpio_write(d_pin, val);
    delay_us(on_us);
    gpio_write(d_pin, 0);
    delay_us(off_us);

    return 0;
}

unsigned int read(){
    if(!d_pin)init();
    return gpio_read(d_pin);
}

```

```

In [74]: # sound on
write(1)
# expect 1 for on
read()

```

Out[74]: 1

```

In [75]: # sound off
write(0)
# expect 0 for off
read()

```

Out[75]: 0

```

In [76]: # test microblaze function
pwm(2,100,1)

```

Out[76]: 0

```

In [77]: # sanity check
import time
def measureTimeOfFn(fn,*args):
    tic = time.perf_counter()
    fn(*args)
    toc = time.perf_counter()
    print("Elapsed time:", toc - tic, "seconds")
def pwmTest(testDurationInSec=1, hz=1, duty=1, val=1, debug=True): # in quiet
    for i in range(testDurationInSec*hz): # forces beep rate for second(s)
        pwm(hz,duty,val)
        if debug: print(i)

```

```

In [81]: # verify the buzzer buzzes for .5 sec
tic = time.perf_counter()
pwm(2,100,1)

```

```

toc = time.perf_counter()
print("Elapsed time:", toc - tic, "seconds")

```

Elapsed time: 0.5026818609994734 seconds

```

In [27]: #headless freq test that counts the number of cycles per sec
# example 10 buzzes for 1 sec
durationInSec = 1
hz=10; duty=50; signal=1;
measureTimeOfFn(pwmTest, durationInSec, hz, duty, signal)

```

0
1
2
3
4
5
6
7
8
9

Elapsed time: 1.017864857999939 seconds

```

In [28]: # headless test to vary duty cycle (volume)
# Choose 10Hz for testing
# sweep duty cycle from 0 to 100
# increments by 10 to save time

# Buzzer will slowly increase in volume.
def sweepDutyCycle(hz=10, duty=101, duration=1, startDuty=0, step=1, debug=True):
    for i in range(startDuty, duty, step):
        pwmTest(testDurationInSec=duration, hz=hz, duty=i, val=1, debug=False)
        if debug: print("duty: %d%%"%i)
    if debug: print("duty cycle test done.")

```

```

In [29]: sweepDutyCycle(hz=10,duty=101,startDuty=0,step=10)

```

duty: 0%
duty: 10%
duty: 20%
duty: 30%
duty: 40%
duty: 50%
duty: 60%
duty: 70%
duty: 80%
duty: 90%
duty: 100%
duty cycle test done.

```

In [32]: # headless test to vary frequency (tone)
# Freq test. Buzzer will slowly increase in tone.
def testBuzzerFreq(hz=100, duty=1, duration=1, startHz=0, step=1, debug=True):
    for i in range(startHz, hz+1, step):
        pwmTest(testDurationInSec=duration, hz=i, duty=duty, val=1, debug=False)
        if debug: print("freq: %dHz"%i)
    if debug: print("Freq test done.")

```

```
def sweepFreq(hz=100, duty=1, duration=1, startHz=0, step=1, debug=True):
    testBuzzerFreq(hz=hz, duty=duty, duration=duration, startHz=startHz, ste
```

In [38]: `sweepFreq(hz=10, startHz=1, step=1, duty=50)`

```
freq: 1Hz
freq: 2Hz
freq: 3Hz
freq: 4Hz
freq: 5Hz
freq: 6Hz
freq: 7Hz
freq: 8Hz
freq: 9Hz
freq: 10Hz
Freq test done.
```

In [53]: `# make button listener`
this is to test the buttons multithreading capability. Each button will tr
import threading
import time
def buzz(hz=2, duty=50, val=1):
 pwm(hz,duty,val)

def btnListener(btns, listenEvent, buzzEvent, closeEvent):
 print("btn listener active.")

 debounce=.5
while(**not** closeEvent.is_set()):
 state = btns.read()

if state & 0b0001: # btn 1
 print("btn 1 pressed")

 time.sleep(debounce)
if state & 0b0010: # btn 2
 print("btn 2 pressed")
 time.sleep(debounce)
if state & 0b0100: # btn 3
 print("btn 3 pressed")
 print("buzzing...")
 buzz()
 print("buzzing done.")
 time.sleep(debounce)
if state & 0b1000: # btn 4
 print("btn 4 pressed")
 time.sleep(debounce)
break;

 time.sleep(.01)

 print("btn listener done.")
 listenEvent = threading.Event()
 buzzEvent = threading.Event()
 closeEvent = threading.Event()

```
btnListenerThread = threading.Thread(target=btnListener,args=(btns, listenEv
btnListenerThread.start()
```

btn listener active.

In [66]: *# 1. This program always starts the server thread listening on port 12345.
2. Button 1 will create a connection to the server sending a '2' indicating
will create a .5 sec tone for each message it receives from the client.
3. Lastly, the 4th button will terminate all connections and stop the prog*

```
import threading
import time
import socket

def buzz(hz=2, duty=100, val=1):
    pwm(hz,duty,val)

def start_server(closeEvent, serverEvent, host="127.0.0.1", port=12345):
    if serverEvent.is_set():
        print("Server already running")
        return
    serverEvent.set()

    server = None
    conn = None

    try:
        server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        server.bind((host, port))
        server.listen(1)

        print("Server is listening on {}:{}".format(host,port))

        conn, addr = server.accept()
        conn.settimeout(1)

        print('Connected by', addr)

        while not closeEvent.is_set():
            try:
                data = conn.recv(1024)
                if not data:
                    print("client disconnected")
                    break
                msg = data.decode(errors="replace")
                print("server received: "+msg)
                buzz(hz=int(msg))
            except Exception as e:
                pass
            conn.close()
            server.close()
            print("server terminated.")
        except Exception as e:
            print("Server error: ", e)
    finally:
```

```

serverEvent.clear()

def client(closeEvent, clientEvent, sendEvent, server_ip="127.0.0.1", port=1
    if clientEvent.is_set():
        print("client already running")
        return
    clientEvent.set()

    s = None
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(1)
        s.connect((server_ip, port))

        print("Connected to {} on port {}".format(server_ip, port))
        while not closeEvent.is_set() and clientEvent.is_set():

            if sendEvent.is_set():
                print("client is sending a 2...")
                s.sendall(b"2") # 2 for 2hz
                sendEvent.clear()
            else:
                pass

        s.close()
        print("client terminated.")
    except Exception as e:
        print("Client error: ", e)
    finally:
        clientEvent.clear()

def btnListener(btns, listenEvent, clientEvent, closeEvent):
    print("btn listener active.")

    debounce=.5
    sendEvent = threading.Event()

    while(not closeEvent.is_set()):
        state = btns.read()

        if state & 0b0001: # button 1
            sendEvent.set()
            if not clientEvent.is_set():
                threading.Thread(target=client, args=(closeEvent, clientEvent
                    time.sleep(debounce)
            if state & 0b1000: # button 4
                print("Terminating conections...")
                closeEvent.set()
                sendEvent.clear()
                time.sleep(1)
                break;
        time.sleep(.01)

    print("btn listener done.")
    closeEvent.clear()

```

```
listenEvent = threading.Event()
buzzEvent = threading.Event()
closeEvent = threading.Event()

# Server listening
threading.Thread(target=start_server, args=(closeEvent,listenEvent,)).start()

btnListenerThread = threading.Thread(target=btnListener,args=(btns, listenEv
btnListenerThread.start()
```

```
btn listener active.
Server is listening on 127.0.0.1:12345
Connected to 127.0.0.1 on port 12345
client is sending a 2...
Connected by ('127.0.0.1', 41848)
server received: 2
client is sending a 2...
server received: 2
client is sending a 2...
server received: 2
Terminating conections...
client disconnected
client terminated.
server terminated.
btn listener done.
```

In []: